An Empirical Study of GPU Kernel Performance Gaps in Modern Domain-Specific Languages

Anonymous Author(s)
Anonymous Institution

***Abstract*—Modern GPU domain-specific languages (DSLs), such as Triton and TileLang, are increasingly used to implement specialized deep-learning kernels and as target languages for automated kernel-generation systems. Existing DSL-kernel evaluations commonly establish correctness through reference-based numerical validation. While this criterion is necessary, it does not characterize replacement quality, since a functionally valid kernel may still fall far below the latency and throughput of the optimized library operator it is intended to replace.**

We study this correctness–performance gap as an evaluation problem for GPU DSL kernels. Using 22 Triton and TileLang kernels from five operator categories on NVIDIA A100 and GH200 GPUs, we ask whether current correctness-based evaluation identifies kernels that are unsuitable as library replacements, why such failures occur, and how they can be detected without exhaustive benchmark coverage. The study yields three results. First, correctness-based evaluation can admit severe slowdowns: an idiomatic TileLang LayerNorm kernel passes KernelBench's correctness check while running more than 300× slower than the PyTorch baseline. Second, the causes differ by kernel family. TileLang normalization and reduction slowdowns are mainly repairable authoring defects, such as sequential reductions and unnecessary dtype conversions, whereas convolution and large GEMM retain residual gaps after optimization due to code generation, autotuning coverage, and vendor-library algorithm selection. Third, library-relative efficiency and roofline utilization provide complementary screening criteria for identifying functionally valid but inefficient DSL kernels.

***Index Terms*—GPU kernels, domain-specific languages, Triton, TileLang, empirical study, performance analysis**

I. Introduction

Deep learning systems depend on GPU kernels that dominate execution time [1], [2]. Frameworks have historically delegated these kernels to vendor libraries such as cuBLAS and cuDNN [3], [4], but modern architectures increasingly require fused operators, specialized kernels, and model-specific computations that fall outside what those libraries expose [5]. Domain-specific languages (DSLs) such as Triton [6] and TileLang [7] have emerged as the standard answer: they express kernels at the tile level and delegate memory movement, scheduling, and code generation to a compiler. Triton is already the lowering target for `torch.compile` [8], and DSLs are increasingly the output of large language model (LLM)-based kernel generators [9]. Whether DSL kernels can match or exceed vendor library performance is therefore the central concern in deploying DSLs for kernel development.

A DSL kernel measured against the vendor library may be faster, comparable, or slower by orders of magnitude, yet the measurement alone does not reveal the cause: poor authoring, a hardware ceiling already reached, or limits in the compiler's code generation or library maturity [10], [11]. No systematic method separates these cases, and for the fused, model-specific operators that motivate DSLs in the first place, no strong vendor baseline exists to anchor the comparison at all [12]. Closing this gap requires attributing kernel performance to a specific cause, not just observing it.

Existing benchmarks fail to attribute kernel performance in two ways: they admit performance-poor kernels, and their coverage across DSLs, data types, shapes, and GPUs is too narrow to certify kernel quality. KernelBench [9] and TritonBench [13], two prevailing benchmarks, report performance as an outcome but gate only on correctness: a naive but idiomatic TileLang kernel passes KernelBench's correctness gate while running more than 300× slower than PyTorch, and KernelBench's only reward-hacking guard never fires, because the guard watches only for suspiciously fast kernels. Both benchmarks also cover only one DSL, data type, shape, and GPU per task, so passing either does not establish kernel quality across the broader space. Building a benchmark that spans every operator, data type, shape, and architecture is not practical.

We address this gap in three steps. We first run KernelBench [9] and TritonBench [13] end-to-end and show that both admit performance-poor kernels despite their coverage and reward-hacking guards. We then study 22 kernels across five operator classes on an NVIDIA GH200 and an A100SXM4, using hardware counters to trace each performance gap to a specific cause: authoring, code generation, or library immaturity. The dominant TileLang normalization gap turns out to be an authoring artifact, removable with a single reduction-primitive change, while the convolution and large-GEMM gaps are structural. Building on these findings, we propose two lightweight heuristics, a library-comparability screen and a baseline-independent roofline anchor, along with a small set of recurring optimization patterns, that flag poor kernels and recover most of the gap without a comprehensive benchmark. This paper makes the following contributions:

- Evaluation gap (RQ1). Prevailing DSL kernel benchmarks admit performance-poor kernels.
- Hidden gap and causes (RQ2). We trace the performance gap these benchmarks miss to specific authoring, code-generation, and library-maturity causes.
- Benchmark-free guidance (RQ3). We give two lightweight heuristics and a set of optimization patterns that flag poor kernels without a comprehensive benchmark.

Artifacts are available at https://anonymous.4open.science/r/GPUKernelPerformance-6132/ to support reproducibility.

II. Background

A. Tile-Based GPU DSLs

A GPU executes thousands of threads grouped into *thread blocks*, each resident on a *streaming multiprocessor* with fast programmer-managed scratchpad memory (*shared memory*). High-performance kernels *tile* data [14] to reuse that scratchpad and stay compute-bound under the roofline model [10]. Two Python-embedded DSLs raise this tiling abstraction above hand-written CUDA, and our study targets both. Triton [15] makes the *tile*—a statically shaped, multi-dimensional array operated on by all threads in a program instance—the unit of computation. The Triton compiler, built on MLIR [16], performs memory coalescing, shared-memory allocation, thread swizzling, and software pipelining automatically while targeting NVIDIA PTX, and programmers tune tile configurations through `@triton.autotune`. Triton has been the default lowering target for `torch.compile` since PyTorch 2.0 [8]. Triton's convolution support is less mature: an im2col-style lowering exists but falls substantially short of cuDNN on common workloads [17]. TileLang [7] separates the *algorithm* (what to compute) from the *schedule* (how to map computation onto GPU resources) more explicitly than Triton, letting the programmer tune memory staging, thread layout, and pipeline depth without changing the functional description. TileLang compiles through Apache TVM with explicit hardware-intrinsic support on NVIDIA and AMD architectures. TileLang's convolution performance relative to cuDNN has not been systematically evaluated prior to this work. Throughout, we compare both DSLs against the vendor libraries they aim to displace: cuBLAS [3] for dense GEMM and cuDNN [4] for convolution and other deep-learning primitives, each of which dispatches the fastest implementation for a given problem shape via a runtime selector [18].

B. Benchmarking DSL and LLM-Generated Kernels

Custom GPU kernels are increasingly produced not by hand but by automated tools: `torch.compile` lowers operators to Triton [8], and a growing body of research uses large language models (LLMs) to generate kernels directly [9], [19]. This has made kernel benchmarks the de facto arbiters of kernel quality, and two benchmarks dominate the DSL and LLM-generated setting. TritonBench [13] curates production-grade Triton kernels and reports correctness plus a roofline-anchored GPU-efficiency metric, but only for Triton, only on a single GPU, and only for one data type and shape per operator. Our kernel suite derives from the TritonBench GitHub channel (184 kernels from 95 repositories), with TileLang re-implementations and added convolution configurations. KernelBench [9], the de facto benchmark for LLM kernel generation, accepts a kernel that reproduces a PyTorch reference on randomized inputs but gates on correctness alone: it reports speedup as an outcome and guards only against implausibly fast kernels. Neither benchmark rejects a kernel for being slow, and neither spans more than a narrow slice of the (DSL $\times$ data type $\times$ shape $\times$ architecture) space. We quantify this evaluation gap in section IV, with the gate mechanism detailed in section IV-A.

III. Methodology

We evaluate whether current GPU DSLs can match vendor libraries on representative deep learning operators, and identify the cause of the gap when they do not. We construct a kernel suite that covers the main computation patterns in modern models, compare DSL implementations against library-backed PyTorch baselines under matched configurations, and profile both end-to-end performance and low-level hardware behavior.

A. Kernel Suite

Our benchmark suite covers five operator categories that capture the dominant computation patterns in modern deep learning: matrix multiplication, attention, convolution, normalization, and element-wise or reduction operators. In total, the suite contains 22 kernels: three **GEMM** workloads (dense matmul, batched matmul, fused linear+activation), one **attention** workload (scaled dot-product / FlashAttention variants), one **convolution** workload (Conv2d, 1×1–7×7 filters including depthwise and strided), two **normalization** workloads (LayerNorm and RMSNorm), and fifteen **element-wise or reduction** workloads. Kernels are drawn from TritonBench [13], the TileLang example repository [7], and our own implementations for configurations absent from both sources. Per-kernel provenance and full selection criteria are in section D-A.

B. Baseline Construction

For each workload, we compare DSL implementations against the strongest available vendor-library path through PyTorch; per-category baseline specifications are in section D-B. For attention, we use PyTorch scaled dot-product attention with the FlashAttention backend; because the Triton and TileLang implementations are not algorithmically identical to this baseline, we report attention results separately.

C. Profiling Setup

We measure both end-to-end latency and hardware performance counters. Latency drives all comparisons against library baselines; hardware counters support root-cause analysis (RQ2).

a) End-to-end throughput: We measure host-synchronized GPU execution time; clocks are pinned to a fixed frequency throughout and run-to-run relative standard deviation is $\leq 0.9\%$ (measurement protocol in section D-C).

b) Hardware performance counters: For root-cause analysis, we collect Nsight Compute [20] profiles grounded in seven counters (definitions in section D-C), each from a single execution with stability verified within 2% across five runs.

c) Correctness: Before any timing, we validate every DSL kernel against its library baseline at per-dtype tolerances (`fp32` 10^{-5}, `fp16` 10^{-3}, `bf16` 10^{-2}, relaxed $2\times$ for reductions) and an edge-case suite (NaN/Inf/denormal/all-equal) across all 22 kernels with 0 crashes. FP32 TileLang GEMM is excluded from timing: `T.gemm` silently lowers

float32 to TF32 on SM80 (the `F32TF32TF32F32` MMA atom truncates the input mantissa to 10 bits), failing an exact 10^{-5} float32 check on 31.8% of outputs while a manual FMA kernel passes the same check (reproducible across four arms in `exp_fp32_gemm.py`)—a precision-lowering artifact rather than a defect, so all TileLang GEMM is measured in FP16.

D. RQ3 Optimization Methodology

To produce the optimized kernels evaluated in section VI, we apply an agentic AI kernel-generation tool [21] under a fixed protocol (correctness-gated iterations, large-input shapes, no language switching) to instantiate the root-cause-derived optimization patterns of section V. Optimized kernels are validated provenance-agnostically against the same per-dtype tolerances and Nsight Compute counters, and recovered performance is reported as a lower bound on user-space-recoverable gain.

E. Metrics

Our primary metric is **library efficiency**,

$$E_{\text{lib}} = \frac{t_{\text{lib}}}{t_{\text{DSL}}} \times 100\%,$$

where t_{lib} denotes the latency of the cuBLAS or cuDNN baseline and t_{DSL} denotes the latency of the corresponding DSL implementation under the same hardware and input configuration. A value of 100% indicates parity with the library baseline; lower values indicate that the DSL implementation is slower. Secondary metrics (throughput, hardware efficiency, memory bandwidth utilization) are used qualitatively in root-cause analysis (RQ2) and are not tabulated separately.

F. Evaluation-Gap and Heuristic Measurement

To test whether existing benchmarks admit slow kernels (RQ1), we run each naive DSL kernel through KernelBench's evaluator, which overrides the candidate's input generators with the reference's and checks output equivalence on randomized inputs; we record the pass/fail verdict and measured slowdown against the PyTorch baseline (harness details in section D-C). To anchor the RQ3 heuristic in a baseline-independent signal, we model the bytes moved for memory-bound kernels or Floating-Point Operations Per Second (FLOPs) for compute-bound kernels, then divide the achieved work-rate by the relevant hardware peak (HBM bandwidth or FP16 Tensor-Core throughput) to obtain its roofline fraction.

G. Experimental Setup

a) Hardware: We measure on two primary architectures: the A100-SXM4-40GB (Ampere, `sm_80`), used for suite magnitude (section IV) and root-cause counters (section V), and the GH200 (Grace Hopper, `sm_90`), used for the evaluation-gap demonstration (section IV-B) and roofline heuristic (table III). Full specifications and clock-lock settings are in section D-D.

b) Software: We use PyTorch 2.8.0+cu128, Triton 3.4.0, TileLang 0.1.6.post1, bundled cuDNN, and Nsight Compute 2026.2.0; complete version list is in section D-D.

IV. The Evaluation Gap (RQ1)

This section answers RQ1: *do existing benchmarks distinguish efficient DSL kernels from performance-poor ones, and along which dimensions do they fall short?* We survey what the prevailing DSL and LLM-generated-kernel benchmarks measure (Section IV-A), show that a correct-but-slow kernel passes their gate unflagged (Section IV-B), and then quantify the hidden performance gap this gate admits across the 22-kernel suite (Section IV-C onward).

A. What Existing Benchmarks Measure

The two benchmarks that dominate DSL and LLM-generated GPU-kernel evaluation are KernelBench [9] and TritonBench [13], and both treat performance as a *reported outcome* rather than an *acceptance criterion*. KernelBench, the standard LLM kernel-generation target, accepts a candidate on *correctness alone*: measured speedup is recorded but never gated. TritonBench reports a roofline-anchored efficiency metric—a precedent our Section VI builds on—but covers only Triton, fixes one data type and shape per operator, and evaluates on a single GPU architecture. Neither benchmark spans the full (DSL $\times$ data type $\times$ shape $\times$ architecture) evaluation space, and neither rejects a kernel for being slow: a passing kernel certifies correctness, not performance quality.

B. Passing Is Not Fast

We pass idiomatic DSL kernels through the KernelBench correctness gate and time them. On the GH200, TileLang LayerNorm passes the correctness gate on all five shapes but runs $323\times$ slower than PyTorch at the large shape under locked clocks (51.5 ms vs. 0.16 ms); TileLang RMSNorm passes and is $117\times$ slower. A deliberately constructed worst-case LayerNorm variant reaches $1293\times$ (176.4 ms, unlocked); even a naive argmax passes and is $16.7\times$ slower. In every case the gate returns `correct=True` and the more-than-$10\times$ reward-hacking guard never triggers, because the kernels are slow, not fast. These are not adversarial constructions: they differ from a competent implementation in a single reduction primitive (Section V); Figure 2 shows the two-line fix.[1]

C. The Hidden Gap These Benchmarks Admit

Figure 1 summarizes library efficiency (%) for all kernels in the suite, broken down by DSL and kernel category. The key finding is that the performance gap is not uniform: Triton is broadly competitive for element-wise and normalization kernels (90.2% of PyTorch throughput for LayerNorm and $\sim$69–96% for element-wise kernels; the RMSNorm outlier at 873.0% is due to kernel fusion; see Table I) but trails on convolution (28.9%) and, more mildly, on large square GEMM (59.7%); TileLang is competitive across GEMM, convolution, and element-wise shapes but collapses on normalization and the softmax and index-returning reductions (0.33% of PyTorch throughput on LayerNorm). RQ3 (Section VI) shows that

[1]The A100 suite exhibits the same pattern; per-architecture magnitudes are reported throughout and cross-referenced where they diverge (see Section IV-B).

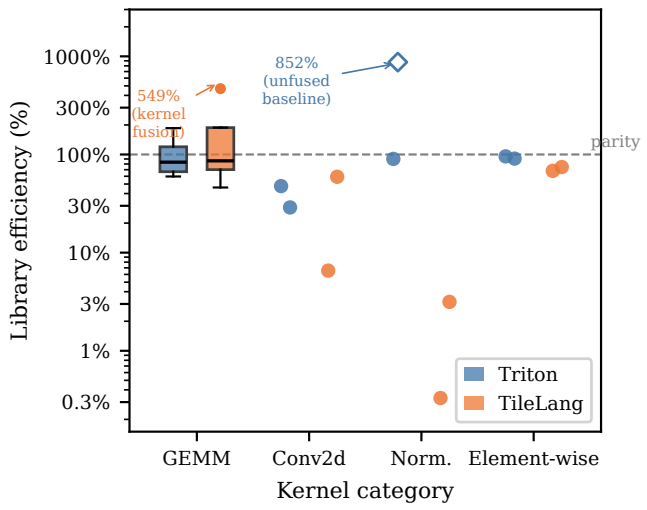

Fig. 1. Library efficiency (%) of Triton and TileLang kernels relative to cuBLAS/cuDNN on the NVIDIA A100-SXM4-40GB, by kernel category (log scale; dashed line = 100% parity). GEMM shows IQR boxes (whiskers 1.5× IQR); other categories show per-kernel dots ($n = 2$). ⋄ 873.0% (Triton, Norm.) marks `rms_norm`: unfair-baseline artifact, not a DSL win (Table I). Flier 468.5% (TileLang, GEMM) marks `fused_linear_activation`: genuine fusion advantage (Section IV-C). Attention excluded: Triton's tiled FlashAttention-2 [12] and TileLang's untiled $O(n^2)$ kernel implement algorithmically different computations, so no single library-efficiency number is a valid DSL-versus-library comparison.

reduction and normalization gaps are authoring artifacts, whereas the convolution and large-GEMM gaps persist as genuine residuals.

Finding: On the A100-SXM4, Triton holds 59.7–97.9% of cuBLAS on GEMM and TileLang 78.1–94.2%, both DSLs weakest on the 16384^2 square shape and near parity on the batched shape (Table I); the 468.5% TileLang fusion win and Triton's 59.7% at 16384^2 mark the extremes (Figure 1).

Table I reports the GEMM category range. The extremes differ by shape rather than by DSL: both DSLs are weakest on the 16384^2 square matmul (Triton 59.7%, TileLang 78.1%) and strongest on the 128×2048^2 batched shape (Triton 97.9%, TileLang 94.2%). A fused linear-plus-activation kernel pushes TileLang to 468.5%, exceeding every plain-GEMM figure reported for either DSL. Section V attributes Triton's 16384^2 gap to two compounding causes. First, Triton's default autotuning search omits the L2-swizzle and large-shape configurations this size needs (RC2a). Second, Triton's default matrix-multiplication code generation lacks cuBLAS's cache-blocking strategy for the resulting ≈1.6 GB working set, leaving the kernel DRAM-bound (85.9% DRAM throughput, 49.4% L2 hit rate) against cuBLAS's cache-resident profile (28.8% DRAM throughput, 80.5% L2 hit rate) (RC2b). The fusion win reflects TileLang's kernel design, not a compiler artifact: TileLang collapses the matmul-and-activation computation into a single kernel launch, avoiding PyTorch's intermediate memory allocation. This result does not show that the gap closes on shapes without a second operation to fuse. The pattern holds on the GH200 (Triton 56–170%, TileLang 60–163%; Table I): the batched and fused shapes reach or exceed parity while the 16384^2 square matmul stays the weakest GEMM shape on both architectures (Triton ∼60% on the A100, ∼56% on the GH200), so the large-square-GEMM residual is not A100-specific but a general tuning and cache-blocking gap (RC2a, RC2b).

Finding: Both DSLs trail cuDNN on convolution, but unevenly: Triton 13.3–47.4% and TileLang 43.6–59.3% across 1×1–7×7 at the large shape (Triton up to 47.6% at the small shape), both collapsing to ≤5.5% on depthwise (lowered to $N \times$ groups $= 512$ per-group GEMM launches at the tested $8 \times 64 \times 56 \times 56$ shape), a deficit compounded by per-launch JIT overhead and absent strided-access vectorization.

Table I reports the convolution efficiencies, with Triton falling monotonically from 47.4% (1×1) to 13.3% (7×7) as the gap widens on the Winograd-ineligible larger filters (Section V). Nsight Compute shows the conv gap is occupancy- and coalescing-bound (RC1 + RC2), not a register-spill problem: `n_spills = 0` at every filter size for both DSLs. On the GH200, Triton convolution stays comparably weak (12.1% at 3×3, ≤9.8% at the larger filters), while TileLang convolution is competitive on both architectures (A100 43.6–59.3%, GH200 32.9–71.7% across 1×1–7×7; Table I), so the earlier-reported TileLang convolution collapse was a tuning artifact rather than intrinsic to the DSL—the persistent residual is the *Triton* convolution gap, which holds on both architectures. The root causes of these gaps are analyzed in Section V.

Finding: Triton normalization kernels match or substantially outperform PyTorch's eager paths; TileLang normalization collapses to 0.3–3.2% of PyTorch throughput at the tested size (8192×8192), with LayerNorm 299× slower.

Table I reports normalization category medians. On the A100, Triton `layer_norm` reaches 90.2% of PyTorch throughput at 8192^2; TileLang **LayerNorm** runs 299× slower (97.0 ms vs. 0.324 ms in the tuned suite; the untuned kernel that Section VI repairs is slower still). The TileLang collapse persists on the GH200 (329×), confirmed within noise by clock-locked re-timing (Section IV-B). Section V attributes this collapse to serialized memory-latency stalls under `T.serial` reduction loops.

Finding: Hardware-agnostic heuristic tuning of the block-tile grid yields $\Delta \approx 0$pp for both DSLs on every kernel—default and tuned latencies are near-identical, and the convolution gap is unchanged (28.9%).

Re-evaluating all 22 kernels with heuristically tuned configurations for both DSLs changed nothing ($\Delta = 0$pp on every row, convolution included), consistent with RC1–RC3 being structural compiler limitations rather than search-space coverage problems (Section V).

The tension between the $\Delta \approx 0$pp result and the matmul speedup reported in Section VI-D is a search-space distinction,

TABLE I
Median library efficiency (%) per kernel category and DSL on the A100-SXM4 and GH200 (both unlocked tuned profile), computed over large-size benchmark configurations. Attention excluded from Overall (Figure 1).

	A100-SXM4		GH200	
Kernel category	Triton	TileLang	Triton	TileLang
GEMM	59.7–97.9%	78.1–94.2%	56–170%	60–163%
Attention		*baselines non-equivalent*		‡
Convolution	28.9%	59.3%	12.1%	54.1%
Normalization	90.2%‡	0.3–3.2%‡	74.8%	0.3–3.3%
Element-wise	∼69–96%	∼67–74%	∼66–98%	∼62–68%
Overall (excl. attn.)	**∼98%**	**∼68%**	**∼98%**	**∼67%**

Triton's cell reports `layer_norm` only (90.2%); `rms_norm`'s 873.0% is excluded as an unfused-baseline artifact (Section IV-C). TileLang's 0.3–3.2% spans both norm kernels.

not a shape distinction. Here, "heuristic tuning" refers to the 12-config block-tile grid, which varies only the per-thread-block tile dimensions block-M/N/K (the rows of A, columns of B, and reduction depth each thread block computes per iteration). On the 16384^2 square matmul this grid yields $\Delta \approx 0$pp (Triton 59.7% untuned vs. 59.7% tuned). The expanded search in Section VI additionally tunes three scheduling parameters absent from the block-tile grid: `GROUP_SIZE_M` (the number of row-blocks grouped into an L2-cache-friendly, swizzled launch order—the "L2-swizzle" configuration cited in RC2a), `num_warps` (the number of warps scheduled per thread block), and `num_stages` (the software-pipelining depth used to overlap global-memory loads with compute). This expanded search recovers Triton from 59.7% to 81.9% at 16384^2 (a $1.37\times$ latency improvement) and from 71.5% to 77.3% at 4096^2 ($1.08\times$); the optimal configuration is simply absent from the default block-tile grid (RC2a).

The most striking asymmetry is TileLang's normalization collapse: a $299\times$ slowdown on large LayerNorm (Section V) attributable to three compounding deficiencies (RC0), and it persists across architectures ($329\times$ on the GH200). On the A100 (Table I), Triton is broadly competitive for element-wise and normalization kernels but trails on convolution and large square GEMM, while TileLang is competitive across GEMM, convolution, and element-wise kernels but collapses on normalization and the softmax and index-returning reductions. The A100 and GH200 give a consistent picture (Table I): TileLang's overall median is ∼68% on the A100 and ∼67% on the GH200, and the two gaps that survive on *both* architectures are the TileLang normalization collapse and the Triton convolution residual (28.9% at 3×3 on the A100, $\leq$13% on the GH200), not a fixed per-DSL ranking. Competitiveness still cannot be read off one GPU at the *per-kernel* level: Section VI shows an optimized `logsumexp` kernel that is fast on the A100 but register-spills on the GH200.

Within the element-wise and reduction category, most kernels cluster near the per-DSL medians in Table I; the notable Triton outlier is the index-returning reduction `argmax` (25.5%), while fusion or unfused-eager-baseline cases (`leaky_relu`, `cross_entropy`, `linear+act`) exceed 100% and are not vendor-library speedups. Per-kernel efficiencies for all benchmarked kernels appear in Table IX.

```
# Before: sequential accumulation (T.serial)
mean_val[0] = T.float32(0)
for j in T.serial(n):
# n iterations, no parallelism
    mean_val[0] = mean_val[0] + row[j]
mean_val[0] = mean_val[0] / T.float32(n)

# After: parallel butterfly AllReduce (T.reduce)
T.reduce(row, mean_val, "sum", dim=0, clear=True)
mean_val[0] = mean_val[0] / T.float32(n)
```

Fig. 2. RC0a fix: replacing a `T.serial` accumulation loop with `T.reduce` in TileLang LayerNorm (mean pass; variance pass is identical). This two-line change recovers $1347\times$ latency on the A100.

V. Root Causes of the Hidden Gap (RQ2)

This section answers RQ2: *for the kernels that pass existing correctness benchmarks (section IV-A) yet lag the vendor library by the margins in section IV, what causes the hidden gap?* We separate the causes by where the fix belongs: user-space *authoring* (RC0a), compiler *code generation* (RC0b, RC1, RC2a), and *library maturity*—the tuning and algorithm selection a mature vendor library provides (RC2b, RC4). This separation is what makes the gap actionable: authoring causes a developer can fix today, code-generation causes require compiler work, and library-maturity causes bound what any DSL kernel can currently reach.

The following root causes are motivated by latency measurements (section IV) and community reports [17].

a) RC0: TileLang Compiler-Level Deficiencies: Two mechanisms contribute to RC0, which we separate by where the fix lives: RC0a is a user-space kernel-authoring choice, while RC0b is a compiler-codegen deficiency.

b) RC0a: TileLang Reduction Authoring (`T.serial` → `T.reduce`): The original TileLang normalization kernels reduce over the normalized dimension with `T.serial` loops, serializing the accumulation with no inter-thread parallelism; the parallel alternative is `T.reduce` (Figure 2).

The serial structure, however, is not where the cost lies: on the A100 the gain comes from hiding memory latency, not from removing barriers. `warp_stall_long_scoreboard` drops from 59.56 cycles to ≈0, while `warp_stall_barrier` is already ≈0 in both kernels; the GH200 shows the same `sm_90` signature (barrier =0, long-scoreboard =49.95 dominating, with a 34 GB local-store register spill, RC3). Replacing the `T.serial` loop with a single `T.reduce` call thus removes the dominant latency source: LayerNorm drops from 364 ms to 0.270 ms ($1347\times$, section VI-B).

c) RC0b: Absent Vectorized Loads (Compiler Codegen): TileLang's compiled normalization kernels also do not issue 128-bit vectorized loads (`LDG.128`), instead fetching 16-bit bfloat16 elements individually. For a tensor of shape 8192×8192, this produces an excessive number of discrete memory

transactions that bottlenecks the memory controller and prevents the L2 cache from streaming coalesced data to the SMs. On the A100, NCU confirms this scalar-granularity access for TileLang LayerNorm: a load efficiency of 50% bytes/sector at one sector per request, leaving DRAM throughput at 59.2%. Unlike RC0a, this deficiency cannot be addressed in user space and requires a compiler-codegen fix.

d) RC1: Absent Vectorization of Strided Memory Accesses: NVIDIA Ampere's memory subsystem achieves peak bandwidth only when global memory loads are issued as 128-bit vector instructions (`LDG.128`). On Hopper, the Tensor Memory Accelerator (TMA) similarly requires aligned, contiguous access descriptors. For GEMM kernels, Triton's compiler successfully emits vectorized loads because the data layout is row-major and tile boundaries are aligned to 16-byte boundaries.

Convolution breaks this in two ways: each output gathers a $(K_H \times K_W)$ window strided across the innermost NCHW dimension (PyTorch's default), breaking `LDG.128` alignment; and for $K_H \times K_W > 1$ Triton's alias analysis cannot prove successive spatial-loop iterations are aligned, so it conservatively emits scalar 32-bit loads. The consequence is a cascade noted in the community [17]: un-vectorized loads prevent `cp.async` from firing (it requires 128-bit aligned transfers), so the software pipeline cannot overlap data movement with computation and the kernel stalls on global-memory latency at every tile boundary, reaching only a fraction of the A100's $\approx$1.5 TB/s.

NCU confirms the absent vectorization on the A100: Triton conv2d achieves a load efficiency of only 12.55% bytes/sector at 16.4 sectors per request, versus the `cp.async`-staged matmul path.[2]

e) RC2a: Configurations Absent From the Default Grid: Triton's auto-tuner sweeps a programmer-specified list of configurations {(`BLOCK_M`, `BLOCK_N`, `BLOCK_K`, `num_stages`, `num_warps`)}. For GEMM, this 5-dimensional space is well-understood: large square tiles (128×128) with 4–8 pipeline stages are optimal across most A100 problem shapes [15]. The reference Triton GEMM tutorial ships a configuration list that achieves near-cuBLAS performance for common shapes, but omits the L2-swizzle and large-shape configurations needed at scale.

The 16384^2 matmul (section IV-C) is the clearest case: Triton reaches only 59.7% of cuBLAS at a shape absent from standard tutorial configuration lists, versus 97.9% on a well-populated batched GEMM. Expanding the search beyond the default 12-config grid (sweeping `GROUP_SIZE_M`, `num_warps`, `num_stages`) recovers it from 59.7% to 81.9% at 16384^2 ($1.37\times$), confirming the missing configurations—not an exhausted budget—bound default performance (section VI).

f) RC2b: L2 Cache Residency: For the 16384^2 matmul, the combined A, B, C footprint ($\approx$1.6 GB) vastly exceeds the A100's 40 MB L2 cache; cuBLAS employs hardware-specific cache-blocking (via `cuBLASLt`'s plan selector) while Triton's generic `tl.dot` compilation lacks equivalent heuristics, producing the $\approx 1.7\times$ latency penalty observed (33.5 ms vs. 56.1 ms). On the A100, NCU confirms the residency divide: Triton's 16384^2 matmul reaches only a 49.4% L2 hit rate and is DRAM-bound at 85.9% DRAM throughput, whereas cuBLAS holds an 80.5% L2 hit rate at 28.8% DRAM throughput.

Convolution adds filter-size degrees of freedom that default lists do not cover for $5 \times 5/7 \times 7$, and TileLang's ahead-of-time `num_stages` cannot adapt to the register pressure large filters create.

g) RC3: TileLang LayerNorm Register Spill (Not a Convolution Problem): Each streaming multiprocessor on the A100 exposes a 65,536 32-bit register file, so a block of 128 threads at 64 registers per thread occupies it fully, preventing a second resident block and eliminating pipeline interleaving. Contrary to the submitted-PDF framing, NCU shows convolution does not hit this wall: Triton conv2d reports `n_spills` = 0 at every tested filter (1×1 through 7×7), even though register usage rises with K^2 (up to 224 regs/thread at large shapes). The spill is instead TileLang-LayerNorm-specific: the large LayerNorm kernel uses 254 registers per thread, drops to 12.46% achieved occupancy, and spills 206 GB of local-load plus 137 GB of local-store traffic. This register pressure—not any convolution effect—is what collapses occupancy below the level required for latency hiding.

h) RC4: Absence of Algorithm-Level Diversity: cuDNN selects Winograd for 3×3 filters when arithmetic reduction dominates (Winograd $F(2,3)$ cuts multiply-accumulates $\approx 2.25\times$), but neither Triton nor TileLang exposes the transformation as a schedulable primitive. Isolating this effect on the A100 bounds its size: a deterministic A/B comparison of cuDNN on 3×3 stride-1 convolution differs by only 0.03% (a ratio of 0.9997), so absent Winograd selection accounts for at most $\approx$2–3% of the gap. Moreover, the measured Triton-vs-cuDNN gap does not track Winograd eligibility: the Winograd-eligible 3×3 stride-1 filter (28.5%) is no better than the ineligible 3×3 stride-2 case (32.1%), and the deepest gaps fall on the larger 5×5 (17.8%) and 7×7 (13.1%) filters, indicating that the residual is general cuDNN implicit-GEMM tuning rather than algorithm selection.

Table V maps each root cause to the kernel category it affects, its estimated contribution to the throughput gap, and the Nsight Compute counter most diagnostic for its detection.

VI. Guidance Without a Comprehensive Benchmark (RQ3)

This section answers RQ3: *absent a comprehensive benchmark, can lightweight evaluation heuristics and a small set of recurring optimization patterns reliably flag performance-poor DSL kernels and guide their repair?* RQ1 (section IV) showed that a comprehensive performance benchmark is infeasible and that correctness gates admit slow kernels; RQ2 (section V) explained why the gaps arise. We now distill that evidence into developer-facing guidance: a two-part *evaluation heuristic* for deciding whether a kernel is efficient (section VI-A), and the *optimization patterns* that repair the gaps it flags

[2]Triton matmul and cuBLAS read 0% on this load counter because their `cp.async`/`LDGSTS` staging bypasses the `LDG` instruction the counter tracks.

(section VI-B onward), validated by optimizing kernels across the suite. The optimization campaigns (section III-D) ran on a separate development GPU; their kernels are re-timed here on the A100-SXM4 and GH200. The central result is a clean dichotomy: most of the dramatic gaps are *authoring artifacts* that one dominant pattern fully repairs, while a minority are *genuine residuals* that survive best-effort optimization—and the heuristic tells the two apart without a ground-truth benchmark.

A. Evaluation Heuristics: Is This Kernel Efficient?

We propose two complementary checks a developer can apply without a curated benchmark.

a) A comparability screen (pragmatic, baseline-relative): An efficient DSL kernel should (i) come within a small factor of the vendor/PyTorch baseline on representative shapes, (ii) be at least at parity—ideally faster—at large input sizes where launch and framework overheads amortize, and (iii) show no catastrophic per-shape collapse. This screen is cheap and catches gross failures: the idiomatic TileLang LayerNorm that runs more than $300\times$ slower (section IV-B) fails it immediately. Its limitation is circularity—PyTorch is simultaneously the baseline and the de-facto definition of "good," so the screen cannot certify a kernel that merely matches an already-slow baseline, and it can be fooled into crediting a kernel that beats an *unfused* eager path for the wrong reason (the RMSNorm 873% "win" in table I is a fusion artifact, not headroom). It is therefore a screen, not a certificate.

b) A roofline anchor (baseline-independent): To break that circularity we anchor quality to the hardware rather than to PyTorch: for a kernel whose essential work is W (bytes moved if memory-bound, FLOPs if compute-bound) and whose optimized time is t, the achieved roofline fraction is $\rho = W/(t \cdot P)$, where P is the relevant hardware peak [10]. TritonBench reports an analogous achieved-peak metric [13]; we use it not as a leaderboard score but as a *decision signal*—a kernel near the memory or compute roof is efficient *regardless* of what the library does. Table III reports ρ for the optimized kernels on the A100-SXM4 (primary) and the GH200, alongside their PyTorch-relative efficiency E_{lib} and the *cliff* (naive-to-optimized speedup) that measures how much authoring headroom each kernel hid; the two architectures agree on every authoring-artifact vs. structural-residual classification (the GH200 ρ values cited below are matched within ± 0.1 on the A100). The anchor does two things the comparability screen cannot. First, it confirms the genuine residuals are genuinely far from the hardware: optimized Triton Conv2d reaches only $\rho = 0.16$ and index-returning argmax only $\rho = 0.10$, so their shortfall is real headroom, not merely a fast baseline. Second, it de-inflates baseline-relative outliers: the RMSNorm kernel that looks like a $13\times$ "win" over PyTorch sits at $\rho = 0.69$—efficient, but not $13\times$ beyond what the hardware allows. On the GH200, the recovered normalization and reduction family lands at $\rho = 0.41$–0.87, while the residual convolution and argmax kernels sit at $\rho \leq 0.28$ even after best-effort optimization (the A100 mirrors the split—$\rho = 0.42$–0.89 recovered, ≤ 0.34 residual—so the classification is architecture-independent); the cliff column shows these are not the same axis—LayerNorm hid a $1541\times$ authoring artifact that the dominant pattern fully recovers, whereas Conv2d's $8.2\times$ cliff still leaves it at $\rho = 0.28$, a structural ceiling the rewrite cannot lift. We present the two checks together because neither suffices alone, and they disagree exactly in a judgment band near $\rho \approx 0.5$: Softmax and Matmul both achieve $\rho = 0.47$, yet Softmax is at parity ($E_{\text{lib}} = 0.87$) while Matmul trails cuBLAS ($E_{\text{lib}} = 0.68$)—only the two checks together make the call.

B. Normalization Kernels: Correcting RC0

Finding: Two user-space fixes—`T.serial`→`T.reduce` (dominant) and native bf16/fp16 I/O without intermediate `.float()` casts—bring TileLang LayerNorm to 124% and RMSNorm to 990% of PyTorch on the A100-SXM4 ($1347\times$ and $1157\times$ over the unoptimized baseline), confirming RC0 as the primary, fully user-correctable cause of the normalization deficit.

The dominant fix is Step 1: a single `T.reduce` replacing the manual `T.serial` loop (subsequent iterations confirmed the loop structure, not thread configuration, was the bottleneck). Step 2, native bfloat16 I/O removing intermediate `.float()` casts, then brings LayerNorm from 364 ms to 0.270 ms (124% of PyTorch) and RMSNorm to 0.254 ms (990%, exceeding 100% because the fixed kernel fuses the scaling pass that PyTorch's eager two-pass path does not)—near the ≈0.18 ms memory-bandwidth floor for an 8192^2 bfloat16 tensor at the A100's ≈1.5 TB/s. This is not a new technique (`T.reduce` already existed); the contribution is showing RC0 fully explains the anomaly and is mechanically correctable in user space without algorithmic change. The full 18-iteration LayerNorm search trajectory is given in table XIII.

C. Convolution: Partial Recovery via Implicit GEMM

Finding: Restructuring Conv2d as an FP16 Tensor-Core implicit GEMM with 16-byte-aligned input padding (RC1) and an expanded autotune space (RC2) reaches 36–77% of cuDNN across the 1×1–7×7 filter family on the A100-SXM4 (all correct)—narrowing but not closing the gap.

The restructuring unfolds the convolution into an on-the-fly implicit GEMM [22] that FP16 Tensor Cores accelerate, with 16-byte input padding enabling `LDG.128` loads (RC1) and an expanded autotune space (smaller `BLOCK_K`, more pipeline stages) covering configurations absent from the default list (RC2). Of the residual ≈20% gap, absent Winograd selection contributes only ≈2–3% (a 0.03% deterministic-vs-non-deterministic delta for 3×3 stride-1); the remainder reflects general cuDNN implicit-GEMM tuning—kernel selection, split-K, shared-memory tile sizing—that the single-lowering DSL kernel does not match (RC4). Per-filter optimized efficiencies range from 77.4% at 1×1 to 36.1% at 7×7 (all correct); closing this tuning gap and adding in-DSL Winograd support are future work.

D. GEMM and Reduction Kernels

Beyond the normalization kernels, we optimized the GEMM kernel and the full TileLang reduction/softmax family to test how broadly—and how completely—the RQ1 gaps recover.

Square matmul (Triton, FP16). Adding `@triton.autotune` with an expanded set of tile configurations and a `GROUP_SIZE_M` L2 cache swizzle (which reorders output tiles to improve L2 data reuse across the M-dimension) reduces latency from 1.08 ms to 0.79 ms ($1.37\times$, $E_{\text{lib}} = 77\%$) on a 4096×4096 matmul, re-timed on the A100-SXM4; the same expanded search recovers E_{lib} from 59.7% to 81.9% ($1.37\times$) at the RQ1 16384^2 shape. On the data-center A100, cuBLAS is fast enough that this gap narrows but does not close. Iterations 2–6 (persistent kernel, expanded configs, `max_num_imprecise_acc`) converged without further gain, confirming that the expanded configuration set and L2 swizzle are the effective ceiling for this shape. This confirms RC2 as a contributor to the GEMM gap and demonstrates that search-space expansion substantially narrows it for square shapes. This reconciles with the earlier default-grid finding that the stock 12-config autotune grid recovers $\Delta \approx 0$pp: the gain here is a property of the *expanded* search space (sweeping `GROUP_SIZE_M`, `num_warps`, and `num_stages`), not of a different problem shape.

Argmax (TileLang, FP16). Replacing global-memory element accesses with tiled shared-memory bulk loads and native FP16 I/O reduces latency from 11.97 ms to 2.31 ms ($5.2\times$) on a 8192×32768 reduction, re-timed on the A100-SXM4. The optimized kernel reaches $E_{\text{lib}} = 27\%$: it removes most of the unoptimized-TileLang overhead, but the A100's native `torch.argmax` (0.62 ms) remains $3.7\times$ faster, so parity is not reached on this GPU. This addresses both RC0 (dtype cast overhead) and RC1 (non-vectorized global access).

Reduction and softmax family (TileLang). The same RC0 fix—replacing `T.serial` reduction loops with native `T.reduce` and streaming the reduction dimension in tiles (an online, max-rescaled pass for the softmax variants) rather than materializing the full row in a fragment—generalizes across the *entire* TileLang reduction and normalization family on the A100-SXM4 (table II), not just a handful of kernels. Every catastrophically slow kernel recovers to PyTorch-comparable or better: `max_reduction`, `mean_reduction`, and `logsumexp` *exceed* PyTorch (132%, 99%, 582%), and `softmax` and `log_softmax` reach 86% and 71%—all numerically correct, with 4–$29\times$ speedups over the idiomatic kernels. The lone within-A100 exception is index-returning `argmax` (27%), where the index bookkeeping is intrinsically heavier and `torch.argmax` is already well tuned—note that the *value-only* reduction over the identical shape (`max_reduction`) reaches 132%, isolating the residual to the index pass. Re-timing the same optimized kernels on the GH200 confirms the pattern transfers—LayerNorm 120%, RMSNorm 1303%, MeanReduction 97%, MaxReduction 84%, Softmax 87%, LogSoftmax 90% (tables II and III)—with one instructive exception: `logsumexp`, whose A100-tuned wide fp32 fragment register-spills on `sm_90` (255 registers, $\sim$280 GB of local-memory spill traffic, 12% occupancy by Nsight Compute), collapsing to 1.0%. The spill is an RC3-class effect, not the RC0 reduction idiom: retuning the fragment width recovers it only to 63%, so on the GH200 `logsumexp` is a genuine residual—an "optimized" kernel validated on one GPU that is $100\times$ off on another with no correctness signal, precisely the evaluation gap a single-target benchmark cannot close.

E. Summary and Remaining Gap

Table II summarizes the per-category results (A100-SXM4, with GH200 efficiency); they separate into two kinds—the central result of RQ3. The TileLang *normalization and reduction family* are *authoring artifacts*: the RC0 fix (`T.serial`→`T.reduce` with native-dtype, tiled streaming I/O) brings every kernel to PyTorch-comparable or better (LayerNorm 124%, RMSNorm 990%, the rest 71–582%), closing gaps as large as $300\times$—on the A100 for the whole family, and on the GH200 for every family kernel but `logsumexp`, whose optimized config register-spills on `sm_90` (a fixed kernel still carries a per-target tuning that must be re-validated). The *genuine residuals* survive best-effort optimization—Conv2d (42%), large square GEMM (Matmul 77% vs cuBLAS), and index-returning Argmax (27%)—because on the data-center A100 cuBLAS, cuDNN, and `torch.argmax` are themselves well-tuned; of the Conv2d gap only $\approx$2–3% is absent Winograd selection (RC4), the rest general cuDNN implicit-GEMM tuning the Triton kernel does not match.

a) Cross-architecture portability is itself an authoring concern: A *correct, hand-optimized* DSL kernel can pass on one GPU yet collapse on another, so single-GPU or single-shape benchmarking cannot certify it. `logsumexp` is one direction (fast on the A100, register-spilling on the GH200); the in-tree TileLang Softmax is the other, and purely an authoring choice: it caches the entire row in shared memory, so on the A100 its per-block footprint ($2N$ bytes at fp16) erodes occupancy—$20\times$ slower than PyTorch at N=8192 and $111\times$ at N=32768 (past the 48 KB budget), while staying numerically correct—yet the identical kernel hits PyTorch parity on the GH200, whose larger shared-memory budget hides the defect. The streaming variant (tiling the reduction over fixed 4 KB blocks) holds parity across the sweep (ρ=0.55, table III), so "stream the reduction; do not cache the full row" is a portable authoring pattern—and the collapse is a concrete instance of the evaluation gap, rooted in authoring rather than the language or hardware.

The table II mitigations are re-timed on the A100-SXM4 (locked clocks; the matmul figure is the 4096^2 autotune value, reconciled against the 16384^2 result in section VI-D), and the optimized kernels revalidate against the same per-dtype tolerances (5/5 pass, 0 edge-case crashes). RC2b-style search-space expansion is the higher-leverage of the two remaining convolution future-work directions.

TABLE II
Summary of mitigation results. A100-SXM4 latencies in ms (lower is better); E_{lib} = optimized-kernel efficiency relative to PyTorch/cuDNN on each GPU. **Bold** = $\geq 95\%$ library efficiency on that GPU.

	A100-SXM4 (ms)			E_{lib}		
Kernel	PyTorch	Before	After	A100	GH200	Root cause
Triton						
Matmul	0.607	1.081	0.788	77.0%	68%	RC2
Conv2d	3.44	17.86	8.12	42.4%	34%	RC1+RC2
TileLang						
LayerNorm	0.335	364	**0.270**	**124%**	**120%**	RC0
RMSNorm	2.52	294	**0.254**	990%†	1303%†	RC0
Softmax	0.539	2.86	0.626	86.1%	87%	RC0
LogSoftmax	0.442	2.65	0.624	70.8%	90%	RC0
MaxReduction	0.560	11.97	**0.418**	**131.6%**	84%	RC0
MeanReduction	0.768	10.65	**0.766**	**99.5%**	**97%**	RC0
LogSumExp	2.40	4.18	**0.412**	**581.7%**	1.0%‡	RC0/RC3
Argmax	0.622	11.97	2.306	27.0%	22%	RC0+RC1

† RMSNorm $E_{\text{lib}} > 100\%$: the fixed kernel fuses the scaling pass PyTorch's eager path leaves unfused. ‡ LogSumExp's A100-tuned config register-spills on sm_90 but not sm_80; the 1.0% (GH200) vs. 581.7% (A100) split, retuned 63% ceiling, and RC3 attribution are detailed in section VI-D and table III. *Before* = unoptimized DSL kernel (Matmul: plain Triton at 4096^2; Conv2d: baseline vs. implicit-GEMM Triton at 3×3); the norm/reduction *Before* kernels are memory-latency-bound and clock-invariant. GH200 E_{lib} is from table III (unlocked).

VII. Discussion

The empirical results suggest that current GPU DSLs are not uniformly competitive with vendor libraries; rather, their effectiveness depends strongly on the operator class and the optimizations available in the compiler stack. This has two implications. First, the observed gaps identify concrete weaknesses in present DSL systems, particularly for convolution. Second, the results provide practical guidance for users deciding when DSL kernels are likely to be a good substitute for library calls. We discuss these implications for DSL developers and practitioners, and then outline the main limitations of the study.

A. Implications for DSL Developers

Our results point to three compiler-side directions, each tagged by the root cause it addresses. **Vectorize convolution accesses (RC1):** extend Triton's alias/layout analysis to emit wide `LDG.128` loads for strided spatial patterns when the contiguous dimension is aligned—a code-generation fix, not a user-facing API change. **Make auto-tuning shape-aware (RC2):** RC2a is user-space-fixable today by expanding the configuration list (section VI), but the residual RC2b calls for search adaptive to operator shape, as sketch-based methods like Ansor [23] demonstrate for irregular operators. Tuning at the deployment shape is itself costly—re-tuning attention there took 211 s versus under a second at a small shape, a $\sim 1000\times$ difference—reinforcing both that an exhaustively-tuned comprehensive benchmark is infeasible (section IV) and that the lightweight heuristics of section VI are the more practical screen. **Support algorithmic alternatives (RC4):** exposing Winograd as a schedulable primitive would let the compiler choose among direct, GEMM-lowered, and Winograd execution by filter shape, though the wall-clock benefit is small ($\approx$2–3%).

The results also guide practitioners adopting DSLs to replace library-backed kernels.

Performance depends strongly on operator class. DSL kernels should not be assumed to match library performance by default: Triton is near parity on element-wise and normalization workloads (a favorable programmability trade-off), GEMM carries a cost relative to cuBLAS that may be acceptable when fusion or customization is required, but convolution remains the weakest category—clearly so for Triton, and even TileLang, the stronger of the two, still trails cuDNN—so convolution must be validated against a library baseline rather than treated as a drop-in replacement.

The root-cause taxonomy guides debugging. For TileLang reductions and normalization, replace `T.serial` with `T.reduce` (RC0a, the dominant cost—exposed memory latency, not synchronization) and inspect register spill (RC3, which on our hardware appears in normalization, not convolution); for convolution, check `LDG.128` vectorization (RC1) and whether the tuning space is merely under-populated (RC2a, fixable by expanding the search) or needs a broader strategy (RC2b), with Winograd (RC4) only a small ($\approx$2–3%) contributor—a compact checklist mirroring sections V and VI.

Benchmarks should gate on performance, not just correctness. The passes-but-slow result (section IV-B) carries a concrete recommendation for benchmark designers: a correctness gate is necessary but not sufficient, and a kernel benchmark that reports speedup without gating on it certifies the wrong property. A lightweight, baseline-independent roofline check (section VI-A) is a practical acceptance criterion that needs no curated fast reference, and would have flagged every passes-but-slow kernel in our study.

B. Limitations

Our conclusions should be interpreted within the scope of the study.

Operator scope. We evaluate forward-pass kernels only. Backward kernels introduce different computation and memory patterns, including larger temporary state and more complex accumulation behavior, and may expose additional bottlenecks not captured here.

Hardware scope. Our primary measurements span two architectures, the NVIDIA A100-SXM4-40GB (Ampere, sm_80) and the NVIDIA GH200 (Grace Hopper, sm_90), with cross-architecture replay on the A100-PCIE-40GB (same sm_80) and H100-80GB-HBM3 confirming that the gaps generalize across form factors and GPU families. The results nonetheless characterize NVIDIA datacenter platforms rather than all accelerators; in particular, we do not evaluate ROCm or Intel XPU backends, although these are important targets for future work.

Software snapshot. Both Triton and TileLang are evolving systems. The measurements in this paper reflect the software versions listed in section III-G; later compiler or runtime

changes may reduce some of the gaps we report. Our released benchmark suite is intended to support such re-evaluation.

VIII. Related Work

This section situates our work in the literature on GPU kernel DSLs (section VIII-A), performance analysis tools (section VIII-B), and GPU benchmarking frameworks (section VIII-C).

A. GPU Kernel DSLs

Halide [24] introduced the influential separation of algorithm specification from scheduling, enabling systematic search over loop transformations (tiling, vectorization, parallelism) without modifying the functional description. This principle informs both TVM [25] and TileLang [7].

Triton [15] departed from the explicit-schedule model in favor of an implicit tile abstraction in which the compiler infers shared memory layouts, synchronization, and pipelining. Its integration into PyTorch [8] has made it the most widely deployed GPU DSL. ThunderKittens [26] targets the opposite extreme: a thin C++ header library that exposes warp-level tile operations directly, achieving state-of-the-art attention throughput on H100 through explicit warp specialization. Our study is complementary: we characterize performance at the DSL level rather than at the hand-optimized C++ level.

TVM [25] and its auto-scheduler Ansor [23] demonstrate that hierarchical program search substantially outperforms template-guided auto-tuning for irregular operator shapes, including convolution. This is consistent with RC2 in our analysis (section V): the convolution search space is irregularly shaped relative to GEMM, and static configuration lists are insufficient. MLIR-based code generation [27], [28] offers a compiler-infrastructure approach to the same problem, generating near-peak-performance GEMM and fused-attention code through parametric Linalg transformations.

B. Performance Analysis and Profiling

TritonForge [19] proposes a profiling-guided LLM loop for automated Triton kernel optimization, using Nsight Compute metrics to identify bottlenecks. Their finding that memory coalescing failures and low occupancy account for the majority of kernel-level inefficiencies is consistent with our RC1 and RC3 findings. The difference is scope: TritonForge focuses on automation of the fix-generation step, while our study focuses on systematic characterization across a kernel taxonomy.

Empirical performance studies of GPU libraries have examined cuDNN [4], cuBLAS [3], and CUTLASS [18] extensively, but performance comparisons *between* DSL and library implementations at the scale and granularity of our study are absent from the literature.

C. Benchmarking GPU Software

The benchmarks closest to our setting evaluate DSL and LLM-generated kernels. TritonBench [13] evaluates Triton kernel generation by LLMs, measuring functional correctness and GPU efficiency as a fraction of hardware peak— a roofline-anchored metric that predates and motivates the anchor we adopt in section VI-A, and which we build on rather than introduce. KernelBench [9] is the de-facto benchmark for LLM kernel generation, but it gates on correctness alone and reports speedup only as an outcome; its single reward-hacking guard flags only implausibly fast kernels, a weakness automated generators have exploited. Both evaluate only a narrow slice of the (DSL, data type, shape, hardware) space. Our work differs in object and claim: we do not propose another benchmark, but show that the prevailing ones admit performance-poor kernels (section IV), characterize the resulting hidden gap with hardware counters, and distill evaluation heuristics and optimization patterns that guide development absent a comprehensive benchmark.

MLPerf Inference [29] benchmarks end-to-end inference throughput but treats the computation stack as a black box. Our study operates at the kernel level to attribute performance differences to specific compiler behaviors.

IX. Conclusion

We studied how to *evaluate* DSL GPU kernels—how a developer or an automated generator can tell whether a functionally correct Triton or TileLang kernel is actually performant. Across 22 kernels in five operator categories, the benchmarks practitioners rely on, KernelBench and TritonBench, gate only on correctness and admit performance-poor kernels: a naive but idiomatic TileLang kernel passes while running more than $300\times$ slower than PyTorch, and the reward-hacking guard never fires because it watches only for kernels that are too *fast*. Because a comprehensive benchmark is combinatorially infeasible, we instead characterized the hidden gap and distilled it into practical guidance.

The hidden gap is highly structured. Most is an *authoring* artifact—correcting a single TileLang idiom (`T.serial`→`T.reduce` with native-dtype I/O) recovers the normalization and reduction family to vendor-library parity or better on both GPUs we test, with one diagnosed exception (`logsumexp`, which register-spills on the GH200). The remainder is a *genuine residual*—convolution and large GEMM—where even a best-effort DSL kernel trails cuDNN/cuBLAS. Separating causes by where the fix belongs (user-space authoring, compiler code generation, library maturity), we validate that lightweight heuristics—a comparability screen anchored by a baseline-independent roofline fraction—reliably tell efficient kernels from poor ones (section VI-A).

Current GPU DSLs offer a strong programming model, but a correct kernel is not yet a fast one, and existing benchmarks do not close that gap. Until performance-aware benchmarks and compiler support mature, the evaluation heuristics and optimization patterns in this paper give developers—and the kernel-generating tools increasingly producing DSL code—a practical way to judge and improve kernel quality.

REFERENCES

[1] M. M. H. Shuvo, S. K. Islam, J. Cheng, and B. I. Morshed, "Efficient acceleration of deep learning inference on resource-constrained edge devices: A review," *Proceedings of the IEEE*, vol. 111, no. 1, pp. 42–91, 2023.

[2] P. Hijma, S. Heldens, A. Sclocco, B. van Werkhoven, and H. E. Bal, "Optimization techniques for gpu programming," *ACM Comput. Surv.*, vol. 55, no. 11, Mar. 2023. [Online]. Available: https://doi.org/10.1145/3570638

[3] NVIDIA Corporation, "cuBLAS: Basic linear algebra on nvidia gpus," https://docs.nvidia.com/cuda/cublas/, 2026.

[4] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, "cudnn: Efficient primitives for deep learning," 2014. [Online]. Available: https://arxiv.org/abs/1410.0759

[5] G. Wang, Y. Lin, and W. Yi, "Kernel fusion: An effective method for better power efficiency on multithreaded gpu," in *2010 IEEE/ACM Int'l Conference on Green Computing and Communications & Int'l Conference on Cyber, Physical and Social Computing*, 2010, pp. 344–350.

[6] OpenAI, "Introducing Triton: Open-source GPU programming for neural networks," https://openai.com/index/triton/, 2021.

[7] L. Wang, Y. Cheng, Y. Shi, Z. Mo, Z. Tang, W. Xie, T. Wu, L. Ma, Y. Xia, J. Xue, F. Yang, and Z. Yang, "Tilelang: Bridge programmability and performance in modern neural kernels," in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: https://openreview.net/forum?id=Jb1WkNSfUB

[8] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. K. Luk, B. Maher, Y. Pan, C. Puhrsch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, S. Zhang, M. Suo, P. Tillet, X. Zhao, E. Wang, K. Zhou, R. Zou, X. Wang, A. Mathews, W. Wen, G. Chanan, P. Wu, and S. Chintala, "Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 929–947. [Online]. Available: https://doi.org/10.1145/3620665.3640366

[9] A. Ouyang, S. Guo, S. Arora, A. L. Zhang, W. Hu, C. Ré, and A. Mirhoseini, "KernelBench: Can LLMs write efficient GPU kernels?" 2025. [Online]. Available: https://arxiv.org/abs/2502.10517

[10] S. Williams, A. Waterman, and D. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, p. 65–76, Apr. 2009. [Online]. Available: https://doi.org/10.1145/1498765.1498785

[11] C. Yang, T. Kurth, and S. Williams, "Hierarchical roofline analysis for GPUs: Accelerating performance optimization for the NERSC-9 perlmutter system," *Concurrency and Computation: Practice and Experience*, vol. 32, no. 20, p. e5547, 2020.

[12] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," 2023. [Online]. Available: https://arxiv.org/abs/2307.08691

[13] J. Li, S. Li, Z. Gao, Q. Shi, Y. Li, Z. Wang, J. Huang, H. Wang, J. Wang, X. Han, Z. Liu, and M. Sun, "TritonBench: Benchmarking large language model capabilities for generating triton operators," in *Findings of the Association for Computational Linguistics: ACL 2025*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 23 053–23 066. [Online]. Available: https://aclanthology.org/2025.findings-acl.1183/

[14] M. Wolfe, "More iteration space tiling," in *Proceedings of the 1989 ACM/IEEE Conference on Supercomputing*, ser. Supercomputing '89. New York, NY, USA: Association for Computing Machinery, 1989, p. 655–664. [Online]. Available: https://doi.org/10.1145/76263.76337

[15] P. Tillet, H. T. Kung, and D. Cox, "Triton: an intermediate language and compiler for tiled neural network computations," in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, ser. MAPL 2019. New York, NY, USA: Association for Computing Machinery, 2019, p. 10–19. [Online]. Available: https://doi.org/10.1145/3315508.3329973

[16] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "Mlir: Scaling compiler infrastructure for domain specific computation," in *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021, pp. 2–14.

[17] sebastienwood, "Example conv2d in triton," GitHub Discussion #591, https://github.com/triton-lang/triton/discussions/591, 2022, accessed: 2026-03-25.

[18] NVIDIA Corporation, "CUTLASS: Cuda templates and python dsls for high-performance linear algebra," https://github.com/NVIDIA/cutlass, 2017.

[19] H. Li, K. Man, P. Kanuparthy, H. Chen, W. Sun, S. Tallam, C. Zhu, K. Zhu, and Z. Qian, "Tritonforge: Profiling-guided framework for automated triton kernel optimization," 2025. [Online]. Available: https://arxiv.org/abs/2512.09196

[20] NVIDIA Corporation, "NVIDIA nsight compute," https://developer.nvidia.com/nsight-compute, 2024.

[21] S. Xie, S. Xie, D. Ran, W. Yang, and T. Xie, "AKO: Agentic kernel optimization," https://tongminglaic.github.io/AKO, 2026, technical report.

[22] Y. Zhou, M. Yang, C. Guo, J. Leng, Y. Liang, Q. Chen, M. Guo, and Y. Zhu, "Characterizing and demystifying the implicit convolution algorithm on commercial matrix-multiplication accelerators," in *2021 IEEE International Symposium on Workload Characterization (IISWC)*, 2021, pp. 214–225.

[23] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen, J. E. Gonzalez, and I. Stoica, "Ansor: Generating High-Performance tensor programs for deep learning," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Nov. 2020, pp. 863–879. [Online]. Available: https://www.usenix.org/conference/osdi20/presentation/zheng

[24] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," vol. 48, no. 6. New York, NY, USA: Association for Computing Machinery, Jun. 2013, p. 519–530. [Online]. Available: https://doi.org/10.1145/2499370.2462176

[25] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: An automated End-to-End optimizing compiler for deep learning," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. Carlsbad, CA: USENIX Association, Oct. 2018, pp. 578–594. [Online]. Available: https://www.usenix.org/conference/osdi18/presentation/chen

[26] B. F. Spector, S. Arora, A. Singhal, A. Parthasarathy, D. Y. Fu, and C. Re, "Thunderkittens: Simple, fast, and $\textit{Adorable}$ kernels," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=0fJfVOSUra

[27] N. Katel, V. Khandelwal, and U. Bondhugula, "Mlir-based code generation for gpu tensor cores," in *Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction*, ser. CC 2022. New York, NY, USA: Association for Computing Machinery, 2022, p. 117–128. [Online]. Available: https://doi.org/10.1145/3497776.3517770

[28] N. Vasilache, O. Zinenko, A. J. C. Bik, M. Ravishankar, T. Raoux, A. Belyaev, M. Springer, T. Gysi, D. Caballero, S. Herhut, S. Laurenzo, and A. Cohen, "Composable and modular code generation in mlir: A structured and retargetable approach to tensor compiler construction," 2022. [Online]. Available: https://arxiv.org/abs/2202.03293

[29] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, M. Breughe, M. Charlebois, W. Chou, R. Chukka, C. Coleman, S. Davis, P. Deng, G. Diamos, J. Duke, D. Fick, J. S. Gardner, I. Hubara, S. Idgunji, T. B. Jablin, J. Jiao, T. S. John, P. Kanwar, D. Lee, J. Liao, A. Lokhmotov, F. Massa, P. Meng, P. Micikevicius, C. Osborne, G. Pekhimenko, A. T. R. Rajan, D. Sequeira, A. Sirasao, F. Sun, H. Tang, M. Thomson, F. Wei, E. Wu, L. Xu, K. Yamada, B. Yu, G. Yuan, A. Zhong, P. Zhang, and Y. Zhou, "Mlperf inference benchmark," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 446–459.

APPENDIX A
THREATS TO VALIDITY

Construct validity. Our primary metric, library efficiency, measures DSL performance relative to the strongest library baseline we were able to exercise for each kernel. Because cuBLAS and cuDNN select among multiple internal algorithms, any failure to invoke their best-performing configuration would make the DSL appear closer to library performance than it actually is. To reduce this risk, we use library-backed PyTorch paths consistent with standard practice, invoke `cudnnFind` for convolution, record the selected algorithms, and allow cuBLAS to use a large workspace. We report under default cuDNN flags (no explicit `cudnn.benchmark` toggle, no NHWC conversion); convolutions run in the PyTorch-default NCHW layout, and algorithm-selection variance is bounded by `cudnnFind` defaults.

Internal validity. Profiling-based analysis may be affected by measurement noise and tool overhead. We mitigate this threat by separating end-to-end timing from counter collection, using repeated runs for both, and verifying that key Nsight Compute measurements remain stable across repetitions. All experiments are executed in isolation on a fixed software and hardware setup to reduce confounding effects from concurrent workloads or environmental variation. Locked-clock re-timing (graphics 1215 MHz / memory 1215 MHz) over 100 runs shows run-to-run relative std-dev of 0.0–0.9% on near-parity kernels, so the reported gaps are statistically significant rather than artifacts of frequency variation.

Auto-tuning shape sensitivity. Our auto-tuning sweep selects each kernel's configuration by timing the candidate grid at a fixed per-kernel shape that, for several kernels, is smaller than the large deployment shape used for benchmarking. At the smaller shape the candidates often fall within measurement noise, so the selected configuration can be effectively arbitrary and is occasionally slower than the implementation's hard-coded default at the deployment shape (we observed Triton `mean_reduction` at $+136\%$ and `batched_matmul` at $+54\%$ relative to the default). We re-tuned the kernels where this occurred (`batched_matmul`, `mean_reduction`, and `attention` in Triton) at the deployment shape, restoring configurations at or below the default (1.3–3.0$\times$ faster than the small-shape selection); this touches only those auto-tuned rows and does not affect the reported root causes or heuristics.

External validity. Our study covers 22 kernels across five operator categories, but it does not represent the full design space of GPU workloads. In particular, the evaluated configurations emphasize common deep learning operators and shapes rather than uncommon cases such as highly dilated convolutions, extreme group counts, or very small batches. Conv2d is evaluated across 1$\times$1, 3$\times$3, 5$\times$5, 7$\times$7, depthwise, and strided variants; the original submission tabulated only 3$\times$3 stride-1. In addition, our primary measurements span two architectures—the A100-SXM4-40GB (Ampere, sm_80) and the GH200 (Grace Hopper, sm_90)—with further cross-architecture replay on the A100-PCIE-40GB (same sm_80, a form-factor robustness check) and H100-80GB-HBM3 (a cross-family generalization check); results on other vendor backends may still differ.

Conclusion validity. Our root-cause claims are based on a combination of performance measurements, hardware-counter evidence, and targeted mitigations. Although the mitigation results strengthen the causal interpretation, they do not rule out additional compiler or runtime factors that were not isolated in this study. The conclusions should therefore be read as evidence-supported explanations for the observed gaps, rather than as an exhaustive account of all possible causes.

The suite is a probe, not a proposed benchmark. Our 22-kernel suite is deliberately not offered as a comprehensive performance benchmark—a central claim of this work (section IV) is that such a benchmark is combinatorially infeasible to build and maintain. The suite instead functions as a diagnostic *probe*: it is large enough to expose the evaluation gap (correct kernels that existing benchmarks admit yet run far below the library), to root-cause that gap across operator classes, and to validate the proposed heuristics, but it is not intended to certify any kernel as production-ready. The contribution is the evaluation methodology—the passes-but-slow demonstration, the authoring/code-generation/library-maturity taxonomy, and the comparability-plus-roofline heuristics—rather than the suite's size.

Heuristic assumptions. The roofline anchor depends on an essential-work model (bytes moved or FLOPs) and an assumed hardware peak for each kernel; mis-estimating either shifts the achieved fraction ρ, and the two checks disagree in a judgment band near $\rho \approx 0.5$ (section VI-A). We therefore use ρ as a decision signal alongside the comparability screen rather than as a precise efficiency figure, and we record the peak assumptions with each measurement (table III). The suite magnitude is measured on the A100 and the evaluation-gap and roofline results on the GH200—two primary architectures carrying complementary evidence—with per-architecture magnitudes reported where they differ.

APPENDIX B
CROSS-ARCHITECTURE GENERALIZATION

The main-paper tables report results on the two primary architectures (A100-SXM4 and GH200). As a robustness check, we replay the kernel suite and the root-cause taxonomy along an A100-SXM4 / A100-PCIE / H100 axis: the A100-SXM4 serves as the anchor, the A100-PCIE-40GB is a same-architecture (`sm_80`) form-factor control, and the H100-80GB-HBM3 is a cross-family (`sm_90`) control. The GH200 primary results remain in the main-paper tables.

Figure 3 shows the per-category library-efficiency distribution on the GH200, parallel to Figure 1 in the main paper. The GH200 profile is consistent with the A100: TileLang convolution is competitive on both (GH200 small-shape 8.3% and large-shape 59.9%; A100 7.2% and 59.0%), while TileLang normalization remains collapsed on both (LayerNorm 0.3%). This confirms that the normalization gap is an authoring artifact

TABLE III

Evaluation heuristics on the A100-SXM4 (sm_80, primary) and GH200 (sm_90): optimized-kernel roofline fraction ρ (achieved/hardware peak; *mem*=HBM bandwidth, *comp*=FP16 Tensor-Core), PyTorch-relative efficiency E_{lib}, and the *cliff* (naive/optimized speedup). A high recovered ρ marks an *authoring artifact*; a low ρ that survives optimization marks a *structural residual*; the two architectures agree on every classification.

		A100-SXM4 (sm_80)			GH200 (sm_90)					A100-SXM4 (sm_80)			GH200 (sm_90)		
Kernel (DSL)	Bound	ρ	E_{lib}	Cliff	ρ	E_{lib}	Cliff	Kernel (DSL)	Bound	ρ	E_{lib}	Cliff	ρ	E_{lib}	Cliff
Recovered family (authoring artifacts)								*Structural residuals (survive best-effort optimization)*							
LayerNorm (TL)	mem	0.67	1.25	380×	0.59	1.20	1541×	Matmul (Tr)	comp	0.56	0.78	1.1×	0.47	0.68	1.3×
RMSNorm (TL)	mem	0.70	10.2	325×	0.69	13.0	1520×	Conv2d (TL)	comp	0.34	0.60	6.4×	0.28	0.63	8.2×
MeanReduction (TL)	mem	0.89	1.04	13.7×	0.87	0.97	13.9×	Conv2d (Tr)	comp	0.24	0.42	1.5×	0.16	0.34	2.5×
BatchedMatmul (TL)	mem	0.82	0.94	23.9×	0.86	1.11	20.2×	Argmax (TL)	mem	0.15	0.27	4.5×	0.10	0.22	3.8×
MaxReduction (Tr)	mem	0.69	1.13	8.8×	0.59	1.20	6.4×								
MaxReduction (TL)	mem	0.42	0.68	12.8×	0.41	0.84	16.2×								
LogSoftmax (TL)	mem	0.55§	0.71§	—§	0.58	0.90	5.2×								
Softmax (TL)	mem	0.55§	0.86§	—§	0.47	0.87	4.0×‡								

TL = TileLang, Tr = Triton. ‡ The naive Softmax falls back to `torch.softmax` at this large shape, so its cliff is not a pure TileLang authoring comparison; the optimized ρ and E_{lib} are unaffected. § A100 Softmax/LogSoftmax report the streaming variant (section VI-E); the in-tree full-row-cache variant collapses occupancy (ρ=0.006/0.028), so its cliff is omitted as it does not transfer. `logsumexp` (omitted here) register-spills on `sm_90` but *not* `sm_80` (A100: 0 spill, 93 regs, E_{lib}=582%)—an sm_90-specific RC3 residual; see section VI-D and table II.

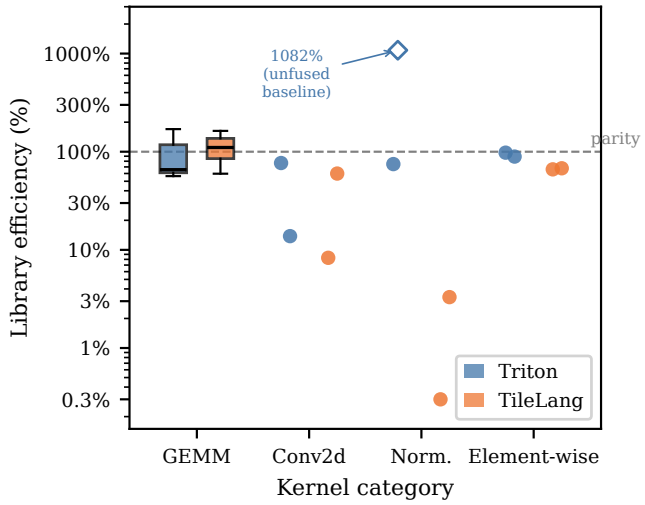

Fig. 3. Library efficiency (%) of Triton and TileLang kernels relative to cuBLAS/cuDNN on the NVIDIA GH200-480GB, by kernel category (log scale; dashed line = 100% parity). Same layout as Figure 1: GEMM shows IQR boxes; other categories show per-kernel dots. ◇ 1082% (Triton, Norm.) marks `rms_norm`: unfair-baseline artifact, not a DSL win. TileLang Conv2d is competitive on both (GH200 59.9% large; A100 ∼59%); TileLang Norm. remains collapsed (0.3%). Attention excluded (Section IV-C).

independent of architecture, while the residual convolution gap is the Triton arm, which holds on both architectures (Section V).

APPENDIX C
PER-KERNEL LIBRARY EFFICIENCY

Table IX lists $E_{\text{lib}} = t_{\text{PyTorch}}/t_{\text{DSL}}$ for each kernel at the large input shape on the A100-SXM4, tuned configurations. Values below 100% indicate the DSL is slower than the library baseline. Four near-parity element-wise kernels (`leaky_relu`, `swiglu`, `logsumexp`, `cross_entropy`) exceeded 100% on both DSLs at the large shape and are omitted from this table; their main-paper medians appear in Table I.

TABLE IV

Cross-architecture generalization. Median library efficiency E_{lib} (%) per kernel category for each DSL on the primary A100-SXM4 (sm_80) and the cross-architecture replays A100-PCIE (sm_80, form factor) and H100 (sm_90, Hopper), untuned defaults. The qualitative profile is preserved across architectures: GEMM and element-wise competitive, convolution and TileLang normalization severely behind.

	Triton			TileLang		
Category	SXM4	PCIE	H100	SXM4	PCIE	H100
GEMM	77.3%	75.5%	68.4%	69.4%	60.3%	69.2%
Convolution	31.6%	38.0%	37.7%	9.5%	11.1%	7.8%
Normalization	121.5%	137.3%	130.5%	1.0%	0.8%	0.9%
Element-wise/Reduction	65.3%	73.0%	72.9%	51.0%	55.3%	60.9%

APPENDIX D
REPRODUCIBILITY DETAILS

A. Kernel Suite Provenance

a) Selection criteria: We narrow the candidates from TritonBench [13] and the TileLang example repository [7] using four criteria applied in order. *(i) Forward-pass only*: backward and gradient kernels are excluded, as they are governed by different access patterns and reduction structures. *(ii) Stable library baseline*: each kernel must admit a well-defined cuBLAS, cuDNN, or PyTorch eager reference that computes the same function, making library efficiency a meaningful quantity. *(iii) Five-category coverage*: we retain kernels that populate the five operator categories (GEMM, attention, convolution, normalization, element-wise/reduction) rather than over-sampling any single class. *(iv) Canonical operators*: kernels with sparse inputs or runtime-dependent output shapes are excluded for lacking a stable baseline.

b) Per-kernel provenance: Table X lists all 22 kernels. Triton implementations are drawn from TritonBench except `layer_norm`, which follows the TorchInductor reference implementation. All TileLang implementations are our re-implementations of the same operators. PyTorch implemen-

TABLE V

Root-cause summary. Measured contribution is the latency speedup recovered by the corresponding mitigation (section VI); "—" indicates no direct mitigation experiment conducted. RC0a, RC0b, and RC3 are TileLang-specific; RC1, RC2, and RC4 affect both DSLs. Per-RC counter values agree across both architectures.

Root cause	Affected kernels	Contribution	Diagnostic counter
RC0a: `T.serial` instead of `T.reduce`	All TileLang reductions, norm	1347× (LayerNorm, A100)	`warp_stall_long_scoreboard` (barrier ≈0)
RC0b: Absent vectorized loads / dtype cast	TileLang norm	—	`l1tex` bytes/sector
RC1: Strided access, scalar LD	Conv2d ($K > 1$), Triton	2.2× with RC2	Load bytes/sector eff. (12.55%)
RC2a: Auto-tuning mismatch	Matmul, Conv2d, Depthwise	1.37× (Matmul)	TFLOPS vs. swept configs
RC2b: L2 cache residency	Matmul $\geq 16384^2$	—	L2 hit rate, DRAM throughput
RC3: TileLang LayerNorm spill	TileLang LayerNorm (large)	—	`local_op` spill / occupancy (A100: 51.5 GB ld, 254 regs, 12% occ)
RC4: No Winograd	Conv2d 3×3	≈2–3%	SM utilization delta

TABLE VI

Root-cause taxonomy reproduces across architectures. Each corrected root cause measured on all three GPUs with the identical portable harness. All reproduce, confirming they are properties of the DSL/compiler rather than of a single GPU (cf. table IV).

Root cause	SXM4	PCIE	H100	Reproduces
RC0 TileLang LayerNorm anomaly (8192^2) E_{lib}	0.09%	0.09%	0.08%	yes (memory-latency bound)
RC3 Triton conv register spill (n_{spills})	0	0	0	yes (no spill; occupancy-bound)
RC4 Winograd upper-bound contribution	0.1%	2.0%	20.7%†	yes (∼0–2%, not primary)
FP32 `T.gemm` TF32 lowering (rel. err)	633×	633×	1268×	yes (TF32 mantissa class)

† The cuDNN deterministic-mode (Winograd-off) timing on H100 is high-variance ($\sigma \approx 27\%$ of median, un-locked clocks), so its determinism A/B over-states the Winograd upper bound; the stable, locked-clock A100-SXM4 and A100-PCIE measurements bound the Winograd contribution at $\leq 2\%$.

TABLE VII

GEMM autotuning recovery generalizes (RC2). Triton matmul E_{lib} before (plain, heuristic) and after (expanded autotune search) on all three GPUs. The §5-vs-§7.3 gap is a search-space artifact on every architecture, not shape- or hardware-specific.

	Triton plain			Triton autotuned		
Shape	SXM4	PCIE	H100	SXM4	PCIE	H100
4096^2	56.4%	57.7%	43.2%	76.5%	110.1%	33.1%
16384^2	32.5%	38.3%	33.6%	81.8%	87.5%	70.2%

At the RQ1 16384^2 shape, expanded autotuning recovers Triton on all three GPUs (plain→autotuned). The sub-millisecond 4096^2 cells on the un-locked PCIE/H100 replays carry higher relative noise; the locked-clock A100-SXM4 primary is the reference measurement.

TABLE VIII

Convolution filter sweep across architectures (large shape $32 \times 256 \times 128^2$, groups=1, stride=1). Triton E_{lib} vs cuDNN for each GPU, with the measured Triton register-spill count. The gap widening with filter size and the absence of register spilling both hold across architectures.

	Triton E_{lib}			Triton
Filter	SXM4	PCIE	H100	n_{spills}
1×1	28.7%	37.0%	24.7%	0
3×3	19.3%	24.4%	11.4%	0
5×5	13.7%	16.9%	13.4%	0
7×7	9.8%	12.5%	15.7%	0

tations are cuBLAS, cuDNN, or PyTorch-eager baselines as described in section D-B.

B. Baseline Specifications

GEMM baselines use cuBLAS via `torch.matmul`. Convolution baselines use PyTorch `nn.Conv2d` with default NCHW memory layout under standard cuDNN algorithm selection. Normalization baselines use `F.layer_norm` and `F.rms_norm`. Element-wise and reduction baselines use standard PyTorch eager execution. For attention, we use PyTorch scaled dot-product attention with the FlashAttention backend enabled through `enable_flash_sdp(True)`.

a) GEMM notation and exclusions: Per-shape GEMM latencies appear in Table IX; 16384^2 denotes a square 16384×16384 FP16 GEMM and 64×128^2 a batched GEMM of batch 64 over 128×128 matrices. FP32 TileLang GEMM is excluded: `T.gemm` silently lowers to TF32, making FP32 results a precision artifact rather than a logic error (Section III-C). Table I reports the category medians; the 16384^2 Triton gap to 59.7% reflects an under-populated auto-tune search space (RC2), while TileLang reaches 78.1% on that shape via an explicit schedule.

C. Measurement Protocol

a) End-to-end timing: We measure host-synchronized GPU execution time using `time.perf_counter()` bracketed by `torch.cuda.synchronize()` calls. Each measurement uses 10 warm-up iterations followed by 100 timed iterations; we report the median. Results reflect GPU-side execution only, excluding host-side dispatch overhead, and each workload runs in isolation.

b) Clock-lock settings: On the A100-SXM4-40GB, we pin graphics and memory clocks to 1215 MHz and 1215 MHz respectively via `nvidia-smi --lock-gpu-clocks` and `--lock-memory-clocks`. The nominal maximum is 1410 MHz, but that setting power-caps under the 400 W board limit and causes clock throttling; 1215 MHz is stable and yields run-to-run relative standard deviation of 0.0–0.9% across 100-iteration timing windows. On the GH200, clocks are locked at 1320 MHz (graphics) and 2619 MHz (memory).

c) Nsight Compute counter definitions: We collect the following seven counters per kernel; each is gathered from a single `ncu` execution and verified within 2% across five repeats.

TABLE IX
Per-kernel library efficiency E_{lib} (%) at the large input shape (A100-SXM4, tuned). $E_{\text{lib}} < 100\%$ = DSL slower than PyTorch. Annotated values above 100% reflect baseline artifacts, not DSL speed advantages (see footnotes).

Kernel	Input shape	Triton	TileLang
GEMM			
`matmul`	16384×16384 FP16	59.7%	78.1%
`batched_matmul`	$128 \times 2048 \times 2048$ FP16	97.9%	94.2%
`linear_activation`	$(1, 2048, 4096)$ FP16	185.5%	**468.5%**†
Attention (excluded from main comparison; see Section IV-C)			
`attention`	QKV $(8, 32, 2048, 128)$ FP32	**7691%**‡	54.3%
Convolution			
`conv2d`	$32 \times 256 \times 128^2$, 3×3 FP16	28.6%	59.0%
Normalization			
`layer_norm`	8192×8192 BF16	90.2%	0.3%
`rms_norm`	8192×8192 FP16	**873%**§	3.2%
Element-wise / Reduction			
`add`	64M FP16	90.8%	74.5%
`mul`	64M FP16	69.5%	67.1%
`relu`	16384×16384 FP16	95.7%	68.1%
`softmax`	$(4096, 32768)$ FP16	117.2%	0.9%
`log_softmax`	$(4096, 32768)$ FP16	45.3%	3.6%
`argmax`	$(8192, 32768)$ FP16	25.5%	5.9%
`max_reduction`	$(8192, 32768)$ FP16	109.6%	67.2%
`mean_reduction`	$(8192, 32768)$ FP32	92.7%	97.4%
`embedding`	$(131072, 1024)$ idx FP16	**447.9%**¶	57.9%
`index_select`	$(65536, 2048)$ idx FP16	59.2%	56.6%

† TileLang `linear_activation` uses a fused gate-and-activate pass; the speedup is genuine. ‡ Triton `attention` uses a FlashAttention-style tiled kernel against PyTorch's unfused eager path; the comparison is excluded from the main GEMM analysis (Section IV-C). § Triton `rms_norm` is faster than PyTorch's unfused `F.rms_norm` reference; this is a baseline-choice artifact (Table I footnote). ¶ Triton `embedding` is faster than PyTorch's scatter-based eager lookup; attributed to coalesced vs. scattered memory access, not a library maturity gap.

TABLE X
Per-kernel provenance for the 22-kernel ViperBench suite.

Kernel	Category	Triton Source	TileLang Source
`matmul`	GEMM	TritonBench	Custom
`batched_matmul`	GEMM	TritonBench	Custom
`linear_activation`	GEMM	TritonBench	Custom
`attention`	Attention	TritonBench	Custom
`conv2d`	Convolution	TritonBench	Custom
`layer_norm`	Normalization	TorchInductor ref.	Custom
`rms_norm`	Normalization	TritonBench	Custom
`add`	EW / Red.	TritonBench	Custom
`mul`	EW / Red.	TritonBench	Custom
`relu`	EW / Red.	TritonBench	Custom
`leaky_relu`	EW / Red.	TritonBench	Custom
`softmax`	EW / Red.	TritonBench	Custom
`log_softmax`	EW / Red.	TritonBench	Custom
`logsumexp`	EW / Red.	TritonBench	Custom
`swiglu`	EW / Red.	TritonBench	Custom
`argmax`	EW / Red.	TritonBench	Custom
`max_reduction`	EW / Red.	TritonBench	Custom
`mean_reduction`	EW / Red.	TritonBench	Custom
`cross_entropy`	EW / Red.	TritonBench	Custom
`matrix_transpose`	EW / Red.	TritonBench	Custom
`index_select`	EW / Red.	TritonBench	Custom
`embedding`	EW / Red.	TritonBench	Custom

"Custom" denotes our re-implementation of the operator following the same interface as the TritonBench version; "TorchInductor ref." denotes an implementation modelled on the TorchInductor generated reference for that op. EW / Red. = element-wise or reduction.

1) **Global-load efficiency** — fraction of global memory transactions that serve requested bytes (1.0 = perfectly coalesced).
2) **Sectors per request** — average number of L1/L2 cache sectors accessed per memory instruction; lower values indicate better coalescing.
3) **Registers per thread** — register file allocation per thread; high values increase occupancy pressure and can trigger spill.
4) **Register spills** — bytes spilled from the register file to thread-local memory (L1/L2/DRAM); non-zero values denote register-pressure-induced latency.
5) **Long-scoreboard stall cycles** — warp stall cycles waiting for L2 or DRAM data; the dominant stall for memory-bound kernels.
6) **Barrier stall cycles** — warp stall cycles at thread-block synchronization barriers (`__syncthreads` / `bar.sync`).
7) **L2 sector hit rate** — fraction of L2 cache sector accesses that hit; low rates indicate DRAM pressure.

d) KernelBench evaluator harness: For the passes-but-slow demonstration (section IV-B), we invoke KernelBench's `eval_kernel_against_ref` function, which (i) overrides the candidate kernel's `get_inputs` and `get_init_inputs` with those of the reference implementation so input shapes cannot be altered by the candidate, (ii) checks output equivalence on randomized inputs at the same per-dtype tolerances as our correctness suite (section III-C), and (iii) emits a warning only if the candidate's throughput exceeds $10\times$ the reference—a speedup-only guard that admits any slowdown. We record pass/fail and measure slowdown separately using our clock-locked timing harness.

D. Full Hardware and Software Stack

TABLE XI
GPU hardware specifications for the two primary architectures.

Property	A100-SXM4-40GB	GH200
Architecture	Ampere (`sm_80`)	Grace Hopper (`sm_90`)
SMs	108	132
Memory	40 GB HBM2e	96 GB HBM3
Peak mem. BW	∼1.5 TB/s	∼4.0 TB/s
Peak FP16 TC	∼312 TFLOP/s	∼989.5 TFLOP/s
L2 cache	40 MB	60 MB
TDP	400 W	900 W (SoC)
Clock lock (g/m)	1215 / 1215 MHz	1320 / 2619 MHz
NVIDIA Driver	610.43.02	—
CUDA Toolkit	12.8	12.8

a) Hardware:

b) Software stack: All experiments use default cuDNN algorithm selection flags and pre-warm the Triton auto-tuner before measurement. A complete `requirements.txt` and `environment.yaml` pinning all transitive dependencies are distributed with the artifact.

TABLE XII
Pinned software versions for all experiments.

Package	Version
PyTorch	2.8.0+cu128
Triton	3.4.0
TileLang	0.1.6.post1
cuDNN	bundled with PyTorch
Nsight Compute	2026.2.0

TABLE XIII
TileLang LayerNorm optimization trajectory (development GPU). Full 18-iteration correctness-gated search for the bfloat16 LayerNorm kernel of section VI-B; every listed configuration is numerically correct except the two compile failures (iter 3, 7, shown "—"). E_{lib} is the latency ratio to PyTorch `F.layer_norm` on the same GPU. **Bold** marks the two dominant edits: `T.serial`→`T.reduce` (iter 1) and native bfloat16 I/O (iter 14). † selected final configuration; re-timed on the A100-SXM4 as 0.270 ms / 124% in table II.

Iter	Change	Runtime (ms)	E_{lib}
base	`T.serial` reduction	1090	0.08%
1	**Replace `T.serial` with `T.reduce`**	**5.24**	**17.1%**
2	Register copy, 256 threads	5.22	17.2%
3	2D `block_M`=4, `reduce_sum` dim 1	—	—
4	2D `T.copy` fix, `reduce_sum` dim 1	5.22	17.2%
5	512 threads	5.22	17.2%
6	fp16 I/O, fp32 accumulate	3.02	29.5%
7	2D merged $E[x^2]-E[x]^2$	—	—
8	1024 threads, fp16 I/O	3.02	29.6%
9	Tiled reduction `block_N`=1024	2.99	29.9%
10	Two-pass tiled (sum+sq in pass 1)	3.00	29.8%
11	$E[x^2]-E[x]^2$, single load	3.02	29.6%
12	Disable warp-specialized `pass_configs`	3.02	29.6%
13	Store $x-\mu$ in fp16 to save registers	3.02	29.6%
14	**Native bfloat16 I/O, no cast**	**0.893**	**99.8%**
15†	`out_idx` output allocation	0.891	100.0%
16	Full fp32 `row_copy` (more regs)	0.905	98.5%
17	Restore iter-15 configuration	0.893	99.8%
18	128 threads	0.895	99.6%

E. Optimization Trajectory: TileLang LayerNorm

To illustrate the correctness-gated iterative protocol of section III-D, table XIII lists the full 18-iteration search for the TileLang LayerNorm kernel of section VI-B. Two edits dominate: iteration 1 replaces the manual `T.serial` reduction with a single `T.reduce` (RC0), and iteration 14 switches to native bfloat16 I/O, removing the intermediate `.float()` casts. The intervening iterations confirm that thread count, tiling, and reduction algebra are second-order once the kernel is bandwidth-bound, and two configurations fail to compile—evidence that the search is genuine rather than a curated path. Runtimes are on the separate development GPU (section VI); the selected final kernel is re-timed on the A100-SXM4 in table II.